Project B - Semester 2, 2026

Table of contents

1 Project B - Semester 2, 2026

2 Univariate Polynomials with Integer Coefficients

(last edit: June 23, 2026)

Due Date & Late Submission Policy: See ECP

Individual work: This is an individual work assignment. Plagiarism will *not* be accepted. Nevertheless, feel free to consult with friends or classmates via Ed and other means about how to go about certain tasks. Note that similar and not identical projects were issued in previous editions of the course. If you wish, you may look at GitHub repos of such projects from previous years (in case students made them public). Do not copy code from those repos directly - but rather use such code bases for help as needed.

Note: The teaching staff will only answer questions (via Ed, consultation hour, or practicals) regarding this assignment up 24 hours before the due date.

Weights and marking criteria: Total number of points: *100*.
There are *10* points for setting up the GitHub repo properly and following instructions for hand-in (including voice recording). The remaining *90* points are for the project tasks.

Submission format: This assignment should be submitted via a GitHub Repo and a PDF file via Blackboard.

Specific instructions for the GitHub repo are below. It is important that the GitHub repo be made **private** and the course user name `MATH2504-staff-2026` be invited as a collaborator. It is also important to name the repo exactly as `<<FIRST NAME>>-<<LAST NAME>>-2504-2026-PROJECTB`. So for example if your name is "Pierre-Simon Laplace", the repo name should be `Pierre-Simon-Laplace-2504-2026-PROJECTB`.

The PDF must have the following components in this order:

1. Your name, student number, and assignment title (Project B - MATH2504 - 2026) on the top.
2. A (clickable) link to your GitHub repo.
3. A screenshot or copy/paste of output for each `example_script_task_i.jl`.
4. A screenshot or copy/paste the output of `test/runtests.jl` for the relevant tasks.

Note: *organize your submission into clearly marked sections. Use headings in the format* `Task i` *or* `Task i.j` *for each part of the assignment.*

Present the new snippets of code that you created and indicate what they do via brief descriptions. It is also a place to answer any questions that require output, or description.

The recommended way to make the PDF file is via a Jupyter notebook where you copy in some code and output into the notebook, and in certain cases use `include()` to run Julia code if appropriate. Also comment on questions in this PDF (e.g. when asked to answer things not via code). If desired keep this jupyter notebook (`.ipynb` file) in the repo. However, this Jupyter notebook will not be checked (only the PDF file), and it is not required to be a "runnable" notebook. In any case, please avoid pixilated screenshots of code (this page may be useful), so for example if you choose to format your PDF file in MS-word (instead of Jupyter), make sure the code is clean, formatted, and readable.

Feedback will be given by the teaching staff by annotating **selected parts** of your PDF file via blackboard. The teaching staff will also inspect your GitHub repo. A very readable and clean PDF file is important. Note that in printing Jupyter to PDF (or exporting to PDF) there may sometimes be excessive white space. Do not worry about this extra white space; it is not a problem.

Note: *as you may have experienced in BigHW, long lines of code in Jupyter notebooks may be truncated by the width of the page when converted to a PDF. Please be aware of this and format your notebook (i.e., split code over multiple lines if needed) as appropriate.*

Marking Criteria:

- *10 points* are allocated for setting up GitHub according to the instructions and following the submission instructions.
- *90 points* are allocated for the polynomial project itself. Points are awarded for completing each of the tasks with full marks given to clean readable code.
- *12 points* are allocated for the *bonus tasks*. No support will be given on Ed for these tasks and tutors will prioritise questions other than those related to the bonus tasks. Bonus marks will be applied on top of your mark for the remainder of the assignment, and your overall mark is capped at 100.
- In general, points **will be** deducted for sloppy coding style. Make sure your code is properly indented, uses sensible and consistent variable names and write code that is in general clean and consistent.

2.1 Setting up your GitHub repo for hand-in (10pts):

- Ideally use the same account you used for BigHW and Project A.
- **Duplicate** the repository https://github.com/dietmaroelz/2504_2026_projectB as described here. You will first create your new repository and then follow the 4 steps under "Mirroring a repository". *Ideally, you would fork the repository, but GitHub does not allow forked repositories to be made private. Do* ***not*** *attempt to fork the repository.*
- Invite the course GitHub user, `MATH2504-staff-2026` as a collaborator and verify this is done (*failure to do so may result in a grade of 0*). You must do this no later than the project due date. It is your duty to check that we accepted the invite (give us 2-5 working days to accept the invite).
- Do **not** make any changes (i.e., commits) to the repo after the project due date.
- Create a local clone of the repo. It is *recommended* that you use the `git` command line to make changes/additions to the assignment and submission. However you are free to use any other mechanism (VS-Code, GitHub desktop, etc). You should make intermediate commits to demonstrate your development process.

Your GitHub repo must be formatted like the original repo.

- Ensure that there aren't excessive files in your submission repo. An exception is perhaps a Jupyter `.ipynb` file if you choose to use it for creating the PDF. Use the `.gitignore` file to ensure `git` does not commit additional files and output files to your `repo`.

2.2 Project tasks (90pts)

Build your project on top of the repo by making modifications, additions, and improvements. Your first goal is to understand the base repository. After that, you will make improvements, additions, and cleanups.

Follow the supplied type names below. These are given so that tasks have backwards compatibility. Note that in an actual project, you would sometimes replace older types with newer ones.

Read all the tasks before starting - only some tasks are dependent on previous ones so they do *not* necessarily have to be done in order.

2.2.1 Task 1 (15pts) - Working in an existing codebase

This task is for getting the repo to run, improving pretty printing, and creating your own example script.

2.2.1.1 Study the Repo

Once you create your own repository, first read the file `README.md`. Then make sure the repository works by running `test/runtests.jl` and `example_script.jl`. At this point simply study the functionality of the repository by looking at the function signatures (not implementations) to understand what the repo can do.

2.2.1.2 Create Example Scripts (*7 points*)

Create your own example script called `example_script_task_1.jl` and include polynomials for illustrating the key functionality of the operations available via the software. Your script should run and demonstrate (using `println` or `@show`) how (*not why*) the code in the repository works. Namely:

1. construct (at least) *three* polynomials (say `f`, `g`, and `h`) using the given constructors for `PolynomialDense`,
2. compute `f + g`, `f * h`, `g * h`,
3. compute `derivative(f * g)` and show it is equal to what is expected when utilizing product rule,
4. compute `(f * h) ÷ h` modulo 5, 17 and 101, and confirm using `mod` (in `polynomial_dense.jl`) that `f` modulo 5, 17, and 101 is returned.
5. compute `gcd_mod_p(f*h, g*h, p)` for `p` equal to 5, 11 and 13.

Basically, `example_script_task_1.jl` is an improvement over the minimally supplied `example_script.jl`.

2.2.1.3 Pretty Printing (*8 points*)

Improve the "pretty printing" of the polynomials by incorporating the following rules:

5. Terms of zero degree (i.e., constants) should exclude `x^0`, e.g. `2` not `2x^0`.
6. Terms with unit degree should exclude the degree, e.g. `3x` not `3x^1`.
7. Excluding the constant term `1`, terms with unit coefficient should exclude the coefficient, e.g. `3x^2 + x` not `3x^2 + 1x`.
8. Use subtraction instead of adding negative terms, e.g. `3x^2 - 2x` not `3x^2 + -2x`.
9. Display the terms in *descending* degree order. $a_n \cdot x^n + a_n \cdot x^{n-1} + \cdots + a_1 x + a_0$.
10. Do not print terms with zero coefficients, e.g. `x^2 + 1` not `x^2 + 0 x^1 + 1`.

Note: feel free to use Unicode characters (but this is not necessary).

11. *Include the output of `example_script_task_1.jl` in your PDF submission via screenshot (or copy/paste).*

2.2.2 Task 2 (20pts) - Utilising Julia's type system

In this task we parameterise the coefficient type to utilize `BigInt`

2.2.2.1 Showing overflow for `PolynomialDense` (*2 points*)

Exact computation requires polynomials with (truly) integer coefficients and thereby we must use `BigInt` rather than (the finitely many integers comprising) `Int`.

1. Create a new script called `example_script_task_2.jl`. Demonstrate using five hard-coded examples (i.e, explicitly write out the polynomials and do not use `rand` to generate them) of `PolynomialDense` addition that will overflow by making the coefficients (*not the degree*) excessively large.

2.2.2.2 Parameterising `Polynomial` (*10 points*)

Notice the struct `Term` is already parameterised and can accept coefficients and degrees that are `BigInt`s. However, the `Polynomial` and `PolynomialDense` types default to using `Int` for both. Your task is to refactor (i.e., modify) these types to be parametric as follows:

2. Refactor `Polynomial` to accept two type parameters called `C` and `D` respectively (named so for coefficient and degree). I.e., we should have

```
abstract type Polynomial{C, D} end
```

3. Refactor all functions in the file `src/polynomial_definitions/polynomial.jl` to account for these new parameters. *Note: you may create a number (or any other concrete type) of type `T` by writing, for example, `T(5)`. The methods `zero(T)` and `one(T)` can create 0 and 1 of type `T`.*

4. Refactor all functions in the directory (a.k.a. folder) `src/basic_polynomial_operations/abstract` to account for these new parameters.

2.2.2.3 Parameterising `PolynomialDense` (*5 points*)

5. At this stage the `PolynomialDense` implementation will no longer work. Refactor the type `PolynomialDense` to accept two type parameters `{C, D}` such that the struct definition starts with

```
struct PolynomialDense{C, D} <: Polynomial{C, D}
    terms::Vector{Term{C, D}}
```

6. Refactor all methods in `src/polynomial_definitions/polynomial_dense.jl` to account for these new parameters. (*Hint: you should use the functions `zero(::Type), one(::Type)` to create 0 and 1*).

7. Refactor all functions in the directory `src/dense` to account for these new parameters.

At this stage we should be able to instantiate a polynomial of type `PolynomialDense{BigInt, Int}`. We need to verify it works correctly. *Note: do not try and instantiate a polynomial of type `PolynomialDense{BigInt, BigInt}` or `PolynomialDense{Int, BigInt}`. These may create absurdly large arrays.*

2.2.2.4 Checking it works (*3 points*)

8. Modify your script `example_script_task_2.jl` to demonstrate that your five original hard-coded examples fail (i.e., produce obviously incorrect output) for `PolynomialDense{Int, Int}` but are correct when using `PolynomialDense{BigInt, Int}` (ie, do not overflow). *Include the output of `example_script_task_2.jl` in your PDF submission.*

9. Refactor the test scripts in the directory `test` to accept your newly parameterised polynomials. Ensure your implementation for `PolynomialDense{Int, Int}` and `PolynomialDense{BigInt, Int}` passes your tests (you should refactor the `test/runtest.jl` file to allow different concrete polynomial types to be tested). *Include the output of the tests in your PDF submission.*

2.2.3 Task 3 (15pts) - Refactoring the data structure used

2.2.3.1 Sparse Polynomials

The current implementation for `PolynomialDense` uses a `Vector` to store the term of degree k at position $k+1$. This is optimal for dense polynomials (those with mostly *nonzero* coefficients) but our factoring algorithms rely heavily on manipulating *sparse polynomials* like $x^n - 1$. In this task we address this deficiency:

2.2.3.2 Constructors (*2 points*)

1. Navigate to the directory `src/polynomial_definitions` and duplicate `polynomial_dense.jl` as `polynomial_sparse.jl`.
2. Inside `src/polynomial_sparse.jl` rename all instances of `PolynomialDense` to `PolynomialSparse` and modify the inner constructor of `PolynomialSparse` so that *zero terms* are excluded from storage. Utilize the max heap (which you can find in `src/utils/heap.jl`) ordered by term degree to accomplish this.
3. Rewrite the *constructors* in `src/polynomial_sparse.jl` to account for the new underlying representation.

2.2.3.3 Operations (*9 points*)

4. Rewrite the *functions* in `src/polynomial_sparse.jl` to account for the new underlying representation. For example, `mod` can no longer utilise array indexing.
5. Navigate to the directory `src/basic_polynomial_operations` and duplicate the directory `dense` as the directory `sparse`.
6. Refactor the *operations* in `src/basic_polynomial_operations/sparse` to account for the new underlying representation. For example, `+` can no longer be done with simple vector addition. To optimise these you may also need to refactor functions in `src/polynomial_sparse.jl`.

*Note: sparse term multiplication and addition can be optimised significantly compared to the abstract implementations. You should override both these functions for your `PolynomialSparse` implementation. You cannot achieve full marks in this section without optimisation to **both** addition and term multiplication. You may need to optimise other helper functions in order to achieve this.*

2.2.3.4 Checking it works (*4 points*)

7. Replicate the `example_script_task_2.jl` for `PolynomialSparse` in a file called `example_script_task_3.jl`.
8. Show (using `BenchmarkTools`) that there are examples where `PolynomialSparse` outperforms `PolynomialDense` for both time and memory usage. *Do this only for cases where `PolynomialSparse` and `PolynomialDense` do not overflow.*
9. Append your code for task 3.8 to `example_script_task_3.jl` and *include the output of the script in your PDF submission.*
10. Describe (at least) three pros and/or cons (*in a table within your PDF*) between the dense and sparse representations.

2.2.4 Task 4 (7pts) - Optimising an algorithm

2.2.4.1 Fixing power (*7 points*)

As discussed in class, if you have a polynomial `f` and raise it to the power `2^n` then instead of doing about `2^n` multiplications you can do about `n` using **repeated squaring**.

1. Re-factor `pow_mod` and `^` into something more efficient by using repeated squaring. Your implementation should be implemented at the abstract level (i.e.,`Polynomial{C, D}`) and should therefore work for both `PolynomialDense` and `PolynomialSparse`.
2. Refactor your algorithms for `pow_mod` and `^` such that `0^n`, `1^n` and `x^n` are constructed without multiplications (i.e., run in O(1)).
3. Create `tests/power_test.jl` containing tests for `pow_mod` and `^` which run on at least 10^2 randomly generated polynomials (*make sure this can be run on your computer in less than 30s*). Ensure your implementations pass these tests.
4. Add this test file to the `test/runtests.jl` file and have your tests print some output demonstrating the correctness of your `pow_mod` and `^` functions. *Screenshot (or copy/paste) the output and add it to your PDF submission'*

2.2.5 Task 5 (8pts) - Implementing $\mathbb{Z}_p$

Notice your polynomial implementation has assumed integer coefficients and (fatally) $\mathbb{Z}[x]$ is *not* a euclidean domain. This means we do not yet have access to `gcd` on polynomials which is a critical component of the factoring algorithm. If we take our coefficients from a *field* like $\mathbb{Z}_p$, for p a prime, then the resulting $\mathbb{Z}_p[x]$ *is* a Euclidean domain. We therefore need to implement some way to work with $\mathbb{Z}_p[x]$.

We have implemented the field-variants of `div`, `gcd`, and `factor` (and so on) as `div_mod_p`, `gcd_mod_p`, `factor_mod_p` (and so on). These new functions *require* an additional input (a prime) and this interface is cumbersome and bad because we do not want to be threading a prime p through all our calculations like this:

```
# Example usage
x = x_poly(PolynomialSparse{Int, Int})
f = (x + 5)^3 * x^2 * (x + 1)^2
g = (x - 1) * (x + 5) * (x + 1)^4
@show gcd_mod_p(f, g, 7)
# output: 5 x^0 + 4 x^1 + 1 x^3 = (x + 5) * (x + 1)^2
```

It is much cleaner to have a polynomial datatype where the "mod p" behavior is "baked in" and automated, eliminating the need to thread p like this:

```
# Example usage
x = x_poly(PolynomialSparse{ZModP{Int, 7}, Int})
f = (x + 5)^3 * x^2 * (x + 1)^2
g = (x - 1) * (x + 5) * (x + 1)^4
@show gcd(f, g)
# output: 5 x^0 + 4 x^1 + 1 x^3 = (x + 5) * (x + 1)^2
```

In this task we implement this improvement. *Note: you should aim to implement all functions in this section as efficiently as you can.*

2.2.5.0.1 Constructors (*2 points*)

1. Create `src/z_mod_p.jl` and inside of this file create a `struct` with the following type signature:

```
struct ZModP{T <: Integer, N} <: Integer
    val::T

    # val   [0,...,N - 1]
    # T is the type of val, N is the prime p in Zp
end
```

Note: that `N` *is a value type and thus it must be specified as a constant, it cannot be generated (e.g. via rand) at runtime. A collection of primes has been provided to you in* `src/utils/sample_primes.jl`*. Use subset of these to work with constants which are primes.*

2. Create inner and outer constructors for this struct. You must be able to construct an object of type `ZModP{T, N}` where `T` is either `Int` or `BigInt` and `N` is some prime. You must you write functions satisfying *all* of the following type signatures:

```
function ZModP{T, N}(a::T) where {T <: Integer, N}
function ZModP(a::T, ::Val{N})::ZModP{T, N} where {T <: Integer, N}
function zero(::Type{ZModP{T, N}})::ZModP{T, N} where {T <: Integer, N}
function one(::Type{ZModP{T, N}})::ZModP{T, N} where {T <: Integer, N}
function zero(a::ZModP{T, N})::ZModP{T, N} where {T <: Integer, N}
function one(a::ZModP{T, N})::ZModP{T, N} where {T <: Integer, N}
```

3. Ensure the constructors in Task 5.2 (above) fail when attempting to create an object of type `ZModP{T, N}` when `N` is *not* a prime. *Use the Primes.jl package that is included in the repository.*

2.2.5.0.2 Field operations and miscellaneous functions (*4 points*)

4. Implement the following convenience and query operations:

```
function show(io::IO, a::ZModP{T, N}) where {T <: Integer, N}
function ==(a::ZModP{T, N}, b::ZModP{T, N})::Bool where {T <: Integer, N}
function ==(a::ZModP{T, N}, b::S)::Bool where {T <: Integer, S <: Integer, N}
function ==(a::S, b::ZModP{T, N})::Bool where {T <: Integer, S <: Integer, N}
```

5. Implement the following basic arithmetic (field) operations:

```
function +(a::ZModP{T, N}, b::ZModP{T, N}) where {T <: Integer, N}
function +(a::ZModP{T, N}, b::S) where {T <: Integer, S <: Integer, N}
function +(a::Int, b::ZModP{T, N}) where {T <: Integer, N}
function -(a::ZModP{T, N}) where {T <: Integer, N}
function -(a::ZModP{T, N}, b::ZModP{T, N}) where {T <: Integer, N}
function -(a::ZModP{T, N}, b::S) where {T <: Integer, S <: Integer, N}
function -(a::S, b::ZModP{T, N}) where {T <: Integer, S <: Integer, N}
function *(a::ZModP{T, N}, b::ZModP{T, N}) where {T <: Integer, N}
function *(a::ZModP{T, N}, b::S) where {T <: Integer, S <: Integer, N}
function *(a::S, b::ZModP{T, N}) where {T <: Integer, S <: Integer, N}
function inv(a::ZModP{T, N})::ZModP{T, N} where {T <: Integer, N}
function ÷(a::ZModP{T, N}, b::ZModP{T, N})::ZModP{T, N} where {T <: Integer, N}
function ÷(a::ZModP{T, N}, b::S)::ZModP{T, N} where {T <: Integer, S <: Integer, N}
function ÷(a::S, b::ZModP{T, N})::ZModP{T, N} where {T <: Integer, S <: Integer, N}
```

Hint: Recall division in a field is simply multiplication by the inverse. See `int_inverse_mod` *in* `src/utils/general_alg.jl`*.*

Note: All the above operations should act as they do in $\mathbb{Z}_p$*. E.g.,* $3 \cdot 2 = 6 \equiv 1 \, (mod 5)$ *should correspond to the code:*

```
a = ZModP(3, Val(5))
@show a * 2 # should print 1
```

6. Implement the additional arithmetic operations

```
function ^(a::ZModP{T, N}, n::S) where {T <: Integer, S <: Integer, N}
function abs(a::ZModP{T, N})::ZModP{T, N} where {T <: Integer, N}
```

7. Implement the following function to convert elements of `ZModP{T, N}` to `Int` or `BigInt`:

```
function (::Type{S})(a::ZModP)::S where {S <: Union{Int, BigInt}}
```

8. Override the function `div` for `Term` (i.e., in `src/term.jl`) to implement it for terms with coefficient type `ZModP`.

9. Utilising your constructors for `ZModP` and the function you created in 7., refactor the function `div_mod_p` for `Term` (i.e., in `src/term.jl`) to simply convert from the input type `Term{C, D}` to `Term{ZModP{C, N}, D}`, then call `div` and finally convert the result back to `Term{C, D}`.

Note: you may make as many additional convenience/helper functions as you wish.

2.2.5.1 Checking that it works (*2 points*)

10. Create `example_script_task_5.jl` and in this file, demonstrate *all* basic modular arithmetic operations working correctly (each at least once).

11. Create `tests/z_mod_p.jl` containing tests which run *all* operations on `ZModP`. Your tests must run on at least 10^3 randomly generated integers in the range $[-10^4, 10^4]$ inclusive and using at least 10 different primes. Utilise the in-built `mod` function on integers to verify correctness of your operations. Ensure your implementations pass these tests.

12. Add this test file to the `test/runtests.jl` file and have your tests print some output demonstrating the correctness of your operations on `ZModP`. *Screenshot (or copy/paste) the output of this and your `example_script_task_5.jl` and add it to your PDF submission.*

2.2.6 Task 6 (7pts) - Implementing $\mathbb{Z}_p[x]$

2.2.6.1 Constructing $\mathbb{Z}_p[x]$ (*5 points*)

Now that we have `ZModP` as a subtype of `Integer`, we can use this as the coefficient type of our polynomials. E.g., `PolynomialSparse{ZModP{Int, 5}, Int}`. *Try experimenting in the REPL with this before attempting the rest of this section.*

Additionally, since we have a baked-in version of $\mathbb{Z}_p$, it is better to re-implement all our functions that were previously named with the suffix `_mod_p` to use this new type.

We also have several functions that are not yet implemented at the abstract level. These are stored in the file `src/basic_polynomial_operations/abstract/polynomial_field_ops.jl`. Your next task is to implement these:

1. Create a folder `src/basic_polynomial_operations/mod_p` and duplicate `src/basic_polynomial_operations/abstr` as `src/basic_polynomial_operations/mod_p/polynomial_field_ops.jl`.

2. Each function in `src/basic_polynomial_operations/mod_p/polynomial_field_ops.jl` takes `::Type{C}` as its first argument. Modify the type signature such that `C` is of type `ZModP{T, N}` (i.e., so we can override it specifically for polynomials over $\mathbb{Z}_p$).

3. Reimplement each function in `polynomial_field_ops.jl` by refactoring the corresponding `_mod_p` version to use your new (better) $\mathbb{Z}_p[x]$ polynomials. *Note: these functions no longer require a prime to be given as input as this is now baked into the type.*

```
function div_rem(::Type{C}, num::P, den::P)::Tuple{P, P} where {P <: Polynomial}
function factor(::Type{C}, f::P)::Vector{Tuple{P,Int}} where {P <: Polynomial}
function dd_factor(::Type{C}, f::P)::Array{P} where {P <: Polynomial}
function dd_split(::Type{C}, f::P, d::Int)::Vector{P} where {P <: Polynomial}
```

Note: it is recommended you refactor functions one at a time and test before moving on to the next one.

4. Write a function for converting between polynomials with terms of type `Term{C, D}` (where `C` is `Int` or `BigInt`) and polynomials with terms of type `Term{ZModP{C, N}, D}` for some given prime `N`.

5. Refactor the functions with suffix `_mod_p` to first convert to a polynomial over $\mathbb{Z}_p[x]$, then call the corresponding `ZModP` function (i.e., the function without suffix `_mod_p`) and then convert back to the initial polynomial type. The functions are as follows:

```
function div_rem_mod_p(num::P, den::P, prime::Int)::Tuple{P, P} where {P <: Polynomial}
function extended_euclid_alg_mod_p(a::P, b::P, prime::Int) where {P <: Polynomial}
function square_free_mod_p(f::P, prime::Int) where {P <: Polynomial}
function factor_mod_p(f::P, prime::Int)::Vector{Tuple{P,Int}} where {P <: Polynomial}
function multiplicity_mod_p(f::P, g::P, prime::Int)::Int where {P <: Polynomial}
function dd_factor_mod_p(f::P, prime::Int)::Array{P} where {P <: Polynomial}
function dd_split_mod_p(f::P, d::Int, prime::Int)::Vector{P} where {P <: Polynomial}
```

Note: Any other functions with suffix `_mod_p` do not need to be refactored as they depend on the functions in Task 6.5.

2.2.6.2 Checking that it works (*2 points*)

6. Verify the correctness of your implementations to the rest of Task 6 by using `tests/runtests.jl` on polynomials with coefficients of type `ZModP{T, N}` for `T` *both* `Int` and `BigInt` and at least 3 primes (and at least 1 prime must be larger than 1000). *Include the output of the tests in your PDF submission.*

2.2.7 Task 7 (18pts) - Factoring with Chinese Remainder Theorem

Note: This is a mastery level task that need only be attempted by those students targeting a seven. It will require extra independent reading of material not covered in class. Sufficient marks have been allocated by this point to earn a (high) six.

1. Create `example_script_task_7.jl`. All *examples* you provide in the remainder of Task 7 must be included in this script, and the output of the script at the end of Task 7 (or however much you are able to accomplish) should be *included in your PDF submission.*
2. Create `src/crt.jl`. All functions you implement in Task 7 should be written in this file unless explicitly stated otherwise.

2.2.7.1 Integer CRT (*4 points*)

2. Implement a function with the following type signature which applies the **Chinese Remainder Theorem** (CRT) over a list of remainders $r1, \ldots, rk$ and (coprime) moduli $m1, \ldots, mk$:

```
function int_crt(rems::Vector{T}, moduli::Vector{T})::S where {T <: Union{Int, BigInt}, S <: Union{Int,
# The output has a different type as you may wish to output a `BigInt` even if `Int`'s are inputted to a
```

4. Demonstrate the correctness of your algorithm with at least 3 examples with 5 or more remainders/moduli.

2.2.7.2 Factorisation with CRT (*14 points*)

5. You have been given factorisation over $\mathbb{Z}_p[x]$. Implement the generalised factoring algorithm over $\mathbb{Z}[x]$ as discussed in Unit 6.
6. Create benchmarks and tests for your implementation. Isolate any bottlenecks and recommend corrections/improvements to address them. Document the technical details of your development process *in your PDF submission.*

*Note: there are nuances in the factoring algorithm we have not covered in class. Due to this you may encounter bugs that you cannot account for. It is fine if your software is **mostly** working but strive to be as correct as possible.*

7. Research Hensel lifting and explain the differences (*in your PDF submission*) between factoring with CRT as discussed in class and factoring utilising Hensel's lemma.

2.2.8 Bonus Tasks (12 points)

Note: *no help will be given via the Ed discussion board or otherwise for bonus tasks.* Bonus tasks may not increase your final mark beyond 100%. E.g., if you get 94/100 for normal tasks and 7/10 for bonus tasks you will get 100/100 (*not 101/100*).

Each bonus task can be completed independent of the others, the order presented here is arbitrary.

2.2.8.1 1. CRT Multiplication (*4 bonus points*)

The current multiplication implementation is not efficient for any concrete subtype of `Polynomial{BigInt, Int}` or `Polynomial{BigInt, BigInt}` because big integer arithmetic is expensive. In this task we reconcile this problem by using CRT to compute products of `Polynomial{BigInt, T}` via cheaper products of `Polynomial{Int, Int}`. *Note: `BigInt` arithmetic is highly optimised in Julia (built on GMP). It is unlikely you will be able to show CRT is faster, but strive to find examples where it is.*

1. Create `example_script_bonus_task_1.jl`. All test cases and benchmarking in the remainder of this section must be included in this script. *Include the output of this script in your PDF submission.*
2. Implement a function to multiply two polynomials using CRT (*you should reuse parts of your code from Task 7*). The product of 2 polynomials must be computed by doing sufficiently many multiplications of each polynomial reduced to $\mathbb{Z}_p[x]$. *Note: you can reuse the primes as global constants from `src/utils/sample_primes.jl`.*

```
function crt_multiplication(f::P, g::P) where {C, D, P <: Polynomial{C, D}}
```

Note: before creating non-trivial tests in the next section, you should test your implementation on trivial examples to verify it is not obviously incorrect.

3. Write at least 10 hardcoded tests demonstrating the correctness of your implementation on non-trivial multiplications (i.e., with `BigInt` coefficients and degree greater than 100) from bonus Task 1.1. *Hint: use polynomials where you know what the coefficients should be after multiplying, e.g., using binomial theorem.*
4. Benchmark (using `BenchmarkTools`) the speed of your CRT multiplication for the examples in bonus Task 1.2 against multiplying the `BigInt`'s directly. Try using both `PolynomialSparse` and `PolynomialDense`.

2.2.8.2 2. FFT/DFT Multiplication (*4 bonus points*)

The naive multiplication algorithm that we have provided (and likely any optimised version you have written) is $O(n^2)$. Modern computer algebra systems use the Fast Fourier Transform (FFT) to compute the Discrete Fourier Transform. This is utilised to multiply both polynomials and (large) integers in $O(n \log n)$. This is one of the *most important algorithms ever developed*.

1. Refactor the existing multiplication of polynomials to use the FFT. In particular, ensure your multiplication uses two different implementations of the FFT to avoid the bit reversal permutation entirely.
2. *In your PDF submission* include an annotated copy of your code, and a description of the technical details of your implementation.
3. Create `example_script_bonus_task_2.jl`. Benchmark using `BenchmarkTools` the speed of your FFT polynomial multiplication algorithm against the naive ones provided.

Note: no marks are awarded for bonus Task 2.2/2.3 if your implementation does not work correctly.

Note: you do not need to learn/implement the truncated FFT (TFFT). The FFT over 2^n terms is sufficient. For those curious, see Paul's paper for an explanation of (TFFT).

2.2.8.3 3. Pseudo-Division in Rings that are *NOT* Euclidean Domains (*4 bonus points*)

As you may have discovered, implementing division over $Z[x]$ is not possible. In particular, we cannot determine a quotient and remainder as $Z[x]$ is not a Euclidean domain. Instead, we have the notion of *pseudo-division. Write all examples for this task in a script called* `example_script_bonus_task_3.jl` *and include the output in your PDF submission.*

1. Write a function `pseudo_div_rem` which returns the *pseudo-quotient* and *pseudo-remainder* of `f` pseudo-divided by `g`. The type signature of the function should be:

```
pseudo_div_rem(f::P, g::P)::Tuple{P, P} where {C, D, P <: Polynomial{C, D}}
```

2. Use the provided `pseudo_gcd` and `square_free` functions (that are currently commented out) in `src/basic_polynomial_operations/polynomial_pseudo_operations.jl` to test that your implementations work.

3. Show that the pseudo-div-rem property (this is similar to the definition of integer division) holds as well as the pseudo-gcd property.

4. Using at least 3 different primes show that the function `square_free` for at least 5 hard-coded examples returns a square free version of your original polynomial (*hint - create polynomials by taking products of irreducibles*) and show that the function `square_free_mod_p(f, prime)` returns the same as `mod(square_free(f), prime)`. *Ensure your script takes no longer than* ***5 minutes*** *to run.*

5. *In your PDF submission* provide a full explanation of the technical details of your pseudo-division algorithm, including why and how it works. Explain the notion of coefficient swell and why your algorithm is slow. Answer whether this issue can be avoided (and why) if we computed this over $\mathbb{Q}[x]$ rather than $\mathbb{Z}[x]$ using exact division over $\mathbb{Q}$.